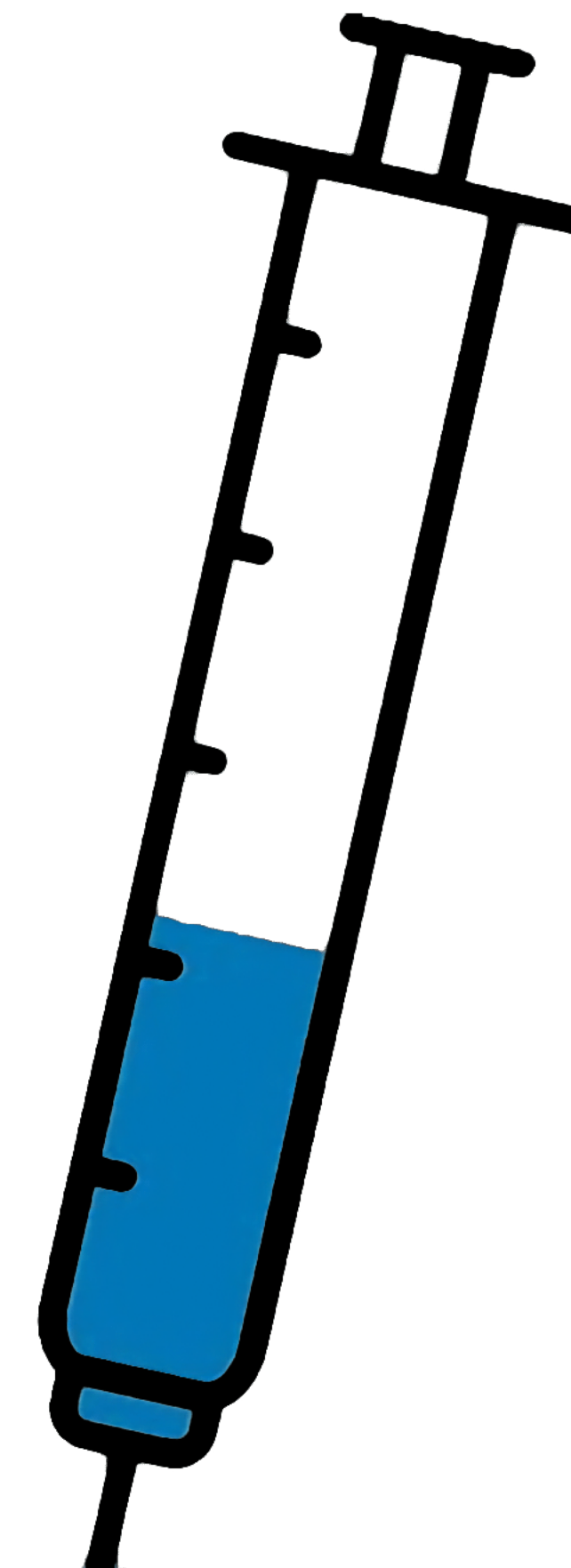


Dependency Injection



Agenda

Проблема без DI

DI как паттерн (ручная DI)

Контейнеры и composition root

Hilt: минимум для проекта

Интеграция с ViewModel + Compose

Что мы знаем

Compose UI - stateless экраны + callbacks

ViewModel - state holder

Repository - data layer (Retrofit + coroutines)

Navigation (sealed routes)

Проблемы

- зависимости растут: API, Repository, кеш, логгер...
- Если создавать всё “внутри классов” - жёсткая связность

**Dependency Injection = объект получает
зависимости извне, а не создаёт их
сам**

Пример

```
class Car {  
    private val engine = Engine()  
  
    fun start() {  
        engine.start()  
    }  
}  
  
fun main(args: Array) {  
    val car = Car()  
    car.start()  
}
```


Пример

```
class Car(private val engine: Engine) {  
    fun start() {  
        engine.start()  
    }  
}  
  
fun main(args: Array) {  
    val engine = Engine()  
    val car = Car(engine)  
    car.start()  
}
```

Пример

```
class AnimeViewModel : ViewModel() {  
  
    private val api = Retrofit.Builder()  
        .baseUrl("https://api.jikan.moe/v4/")  
        .addConverterFactory(GsonConverterFactory.create())  
        .build()  
        .create(JikanApi::class.java)  
  
    private val repository = AnimeRepository(api)  
  
    fun search(query: String) {  
        // repository.searchAnime(query)  
    }  
}
```


Пример

```
class AnimeViewModel(  
    private val repository: AnimeRepository  
) : ViewModel() {  
  
    fun search(query: String) {  
        // repository.searchAnime(query)  
    }  
}
```

Что изменилось

- класс не создаёт зависимости сам
- класс занимается своей задачей
- зависимости можно заменять
- код проще тестировать, расширять, переиспользовать

Без DI

- `class Vm { val repo = JikanRepo(NetworkModule.api) }`
- Проблемы:
 - нельзя подменить repo
 - сложно тестировать
 - сложно контролировать конфигурацию (baseUrl, interceptors)

Why DI?

- DI даёт:
 - тестируемость
 - заменяемость реализаций
 - Масштабируемость
 - контроль lifecycle объектов

Почему не “object Singleton”

- object = глобальное состояние
- Минусы:
 - сложно тестировать (глобальные зависимости)
 - сложно менять окружение (staging/prod)
 - скрытые зависимости - код трудно читать
- DI решает это явными конструкторами

Ручной DI

- Ручная DI = создаём зависимости вручную
- Но в одном месте, через контейнеры/фабрики
- Собираем зависимости в одном месте и передаём в ViewModel

Пример

// Container of objects shared across the whole app

```
class AppContainer {
```

// Since you want to expose userRepository out of the container, you need to satisfy

// its dependencies as you did before

```
private val retrofit = Retrofit.Builder()
```

```
    .baseUrl("https://example.com")
```

```
    .build()
```

```
    .create(LoginService::class.java)
```

```
private val remoteDataSource = UserRemoteDataSource(retrofit)
```

```
private val localDataSource = UserLocalDataSource()
```

// userRepository is not private; it'll be exposed

```
val userRepository = UserRepository(localDataSource, remoteDataSource)
```

```
}
```

// Custom Application class that needs to be specified

// in the AndroidManifest.xml file

```
class MyApplication : Application() {
```

// Instance of AppContainer that will be used by all the Activities of the app

```
val appContainer = AppContainer()
```

```
}
```

Composition root: где “создаём мир”

- В Android это обычно:
 - Application / DI контейнер
 - либо Activity (если ручная DI)
- Идея: создание вынесено из бизнес-логики

DI контейнер: зачем нужен

- Контейнер автоматизирует:
 - создание объектов
 - переиспользование (singleton)
 - привязку жизненного цикла (scopes)

Hilt: что это и почему его выбирают

- Hilt - библиотека DI для Android, уменьшает boilerplate ручной DI
- рекомендованный Jetpack DI
- Даёт контейнеры для Android-классов и управляет их lifecycle

Что делает Hilt “под капотом”

- Строит дерево зависимостей
- Делает compile-time проверку
- Создаёт “containers” под Android классы

Минимальная настройка Hilt

- Подключить Gradle plugin и зависимости Hilt
- `@HiltAndroidApp` на Application
- `@AndroidEntryPoint` на MainActivity

Минимальный набор аннотаций Hilt

- `@HiltAndroidApp` (Application)
- `@AndroidEntryPoint` (Activity/Fragment/...)
- `@HiltViewModel` + `@Inject` constructor(...)
- `@Module` + `@Provides`/`@Binds` + `@InstallIn(SingletonComponent::class)`

@HiltAndroidApp

- Аннотация Application:
 - генерирует базовый контейнер приложения
 - включает Hilt граф

Пример модуля

```
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides fun provideApi(...): JikanApi = ...
}
```

ViewModel + Compose

- `@HiltViewModel`
- `@Inject constructor(repo: AnimeRepository)`
- Hilt сам создаст ViewModel и отдаст её
- Для Navigation Compose:
- `val vm = hiltViewModel<MyVm>()`

Scopes

- SingletonComponent - “на всё приложение”
- Зачем:
 - один Retrofit/OkHttp на весь процесс
 - один Repository на всё приложение